

RASPID: Reasoning-Aware Steering with PID Control for Efficient Language Model Generation

Anonymous Authors
Affiliation
email@domain.com

Abstract

Large Language Models (LLMs) employing extended chain-of-thought (CoT) reasoning, such as OpenAI’s o1-series, demonstrate remarkable capabilities in complex reasoning tasks. However, these models often generate excessive redundant tokens during reasoning, leading to increased inference costs and latency without corresponding performance gains. We introduce RASPID (Reasoning-Aware Steering with PID control), a training-free method that dynamically reduces redundant reasoning tokens while maintaining or improving accuracy. Our approach combines chunk-level classification of reasoning utility with PID-controlled activation steering to provide fine-grained, real-time control over token generation. Unlike existing methods that apply static steering coefficients, RASPID adapts its intervention strength based on real-time redundancy detection. Extensive experiments on GSM8K demonstrate that RASPID achieves 32% token reduction while improving accuracy by 6% over baseline generation. Our method provides a practical solution for deploying reasoning-capable LLMs with improved efficiency.

1 Introduction

The emergence of reasoning-capable Large Language Models (LLMs) such as OpenAI’s o1-series has marked a significant milestone in artificial intelligence, demonstrating human-like problem-solving abilities across diverse domains including mathematics, coding, and scientific reasoning. These models achieve their impressive performance through extended chain-of-thought (CoT) reasoning processes that dynamically scale test-time computation, allowing for iterative refinement and verification of solutions.

However, this extended reasoning paradigm introduces substantial computational overhead. The autoregressive nature of LLMs means that generation is typically memory-bound, with KV cache size growing linearly with sequence length. More critically, recent studies have revealed that these models often generate excessive redundant tokens during reasoning, leading to what we term the “token wastage problem” – where models continue generating long reasoning chains even after arriving at correct solutions, or engage in repetitive verification loops that degrade rather than improve performance.

Consider a simple mathematical question like “What is $1 + 1$?” A reasoning-capable model might generate hundreds of tokens exploring edge cases, alternative number systems, and extensive verification steps, when a direct answer would suffice. This redundancy not only increases inference costs but can actively harm performance by introducing unnecessary complexity and potential for errors.

Current approaches to addressing this problem fall into two main categories: prompt engineering techniques that attempt to encourage brevity through instructions, and static activation steering methods that apply fixed interventions. However, prompt engineering has proven ineffective at the deep behavioral level required for meaningful token reduction, while static steering methods lack the adaptability needed for dynamic reasoning scenarios.

We propose RASPID (Reasoning-Aware Steering with PID control), a novel approach that addresses these limitations through three key innovations:

1. **Dynamic Redundancy Detection:** A chunk-level classifier that identifies redundant vs. useful reasoning in real-time during generation

2. **PID-Controlled Steering:** A feedback control system that dynamically adjusts steering strength based on detected redundancy levels
3. **Training-Free Deployment:** No model fine-tuning required, enabling immediate application to existing reasoning models

Our method draws inspiration from control theory, treating language generation as a dynamic system that can be steered toward more efficient reasoning paths through continuous feedback. The PID controller provides stable, responsive control without the oscillations that can occur with simpler intervention schemes.

2 Related Work

2.1 Chain-of-Thought Reasoning in LLMs

Chain-of-thought prompting (?) has become a fundamental technique for eliciting reasoning capabilities from large language models. Recent advances have led to models that can perform extended reasoning automatically, most notably OpenAI’s o1-series models (?) and open-source alternatives like DeepSeek-R1 (?) and QwQ. These models demonstrate impressive capabilities on challenging reasoning benchmarks but often generate lengthy reasoning traces.

2.2 Activation Steering and Representation Engineering

Activation steering, also known as representation engineering (?), manipulates model behavior by intervening in intermediate representations. Recent work has shown that concepts like truthfulness, helpfulness, and various behavioral traits can be controlled through linear directions in activation space. Our approach extends this paradigm to reasoning efficiency, providing dynamic rather than static control.

2.3 Concurrent Work: SEAL

The SEAL method (?) addresses similar challenges by categorizing reasoning into execution, reflection, and transition thoughts, then applying steering to reduce excessive reflection and transition. While SEAL demonstrates effectiveness, it relies on predefined thought categories and applies static steering coefficients. Our approach differs by providing dynamic, adaptive control based on real-time redundancy detection.

2.4 Efficiency in Language Model Inference

Various approaches have been proposed to improve LLM inference efficiency, including speculative decoding, early stopping, and adaptive computation. Our work is complementary to these approaches, focusing specifically on reducing redundant reasoning content rather than accelerating the generation process itself.

3 Problem Formulation

3.1 The Token Wastage Problem

We define the token wastage problem as the tendency of reasoning-capable LLMs to generate excessive tokens that do not contribute meaningfully to problem-solving accuracy. This manifests in several ways:

1. **Redundant Verification:** Repeatedly checking already-correct solutions
2. **Excessive Exploration:** Considering irrelevant alternative approaches
3. **Verbose Explanation:** Using unnecessarily complex language for simple concepts
4. **Circular Reasoning:** Getting trapped in repetitive thought loops

3.2 Quantifying Redundancy

We formalize reasoning redundancy through a binary classification framework. Given a reasoning trace divided into chunks of tokens, we define:

- **Useful chunks:** Token sequences that advance problem-solving progress
- **Redundant chunks:** Token sequences that do not contribute to or may hinder solution quality

Our goal is to maximize the proportion of useful chunks while minimizing redundant ones, subject to maintaining or improving final answer accuracy.

3.3 Dynamic Control Objective

Unlike static intervention methods, we aim to provide adaptive control that responds to the current state of reasoning. Formally, we seek a control function $\alpha(t)$ that determines steering strength at generation step t based on:

- Current redundancy level $p_{\text{redundant}}(t)$
- Historical redundancy patterns
- Target redundancy threshold τ

The control objective is to maintain $p_{\text{redundant}}(t) \approx \tau$ while minimizing intervention when the model is generating useful content.

4 Method

RASPID consists of three main components: a chunk-level redundancy classifier, a control vector for steering model behavior, and a PID controller that dynamically adjusts steering strength. Figure ?? illustrates the overall architecture.

4.1 Chunk-Level Redundancy Classification

4.1.1 Data Preparation

We begin by collecting reasoning traces from the target model on mathematical reasoning problems. These traces are manually labeled at the chunk level as either “required” (useful for problem-solving) or “redundant” (unnecessary or potentially harmful).

For our experiments, we use 1000 reasoning chains from GSM8K problems, segmented into overlapping chunks of 16-24 tokens. The labeling process focuses on identifying:

- **Required thoughts:** Direct problem-solving steps, calculations, and logical deductions
- **Redundant thoughts:** Excessive verification, repetitive explanations, and tangential explorations

4.1.2 Classifier Architecture

We train a binary classifier using embeddings from the model’s intermediate layers. Specifically, we:

1. Extract hidden states from layer 20 (empirically determined optimal layer)
2. Average-pool hidden states across tokens within each chunk
3. Train an SGD classifier with logistic loss on the resulting embeddings

The classifier is trained to predict $P(\text{redundant}|\text{chunk_embedding})$, providing a real-time redundancy score during generation.

4.1.3 Chunk Size Optimization

We evaluate multiple chunk sizes (16, 24 tokens) to balance granularity with classification accuracy. Smaller chunks provide finer control but may lack sufficient context, while larger chunks may be too coarse for precise intervention.

4.2 Control Vector Construction

Following the representation engineering paradigm, we construct a control vector that encodes the direction in activation space corresponding to “useful vs. redundant” reasoning.

4.2.1 Activation Collection

For each text type (required/redundant), we:

1. Pass the text through the model to obtain hidden states
2. Extract representations from the target layer (layer 20)
3. Compute mean-pooled representations across sequence length

4.2.2 Vector Computation

The control vector is computed as:

$$\mathbf{v}_{\text{control}} = \text{mean}(\mathbf{v}_{\text{required}}) - \text{mean}(\mathbf{v}_{\text{redundant}}) \quad (1)$$

This direction in activation space points from redundant toward required reasoning patterns.

4.3 PID-Controlled Dynamic Steering

The core innovation of RASPID is the use of a PID controller to dynamically adjust steering strength based on real-time redundancy detection.

4.3.1 PID Controller Design

Our PID controller takes the form:

$$\alpha(t) = K_p \times e(t) + K_i \times \int e(\tau) d\tau + K_d \times \frac{de(t)}{dt} \quad (2)$$

Where:

- $e(t) = p_{\text{redundant}}(t) - \tau$ is the error term
- $\tau = 0.5$ is the target redundancy threshold
- K_p, K_i, K_d are tunable gains

The controller only activates when redundancy exceeds $\tau + \text{margin}$ to avoid unnecessary intervention during useful reasoning.

4.3.2 Controller Parameters

Based on empirical tuning, we use:

- $K_p = 0.05$ (proportional gain)
- $K_i = 0.001$ (integral gain)
- $K_d = 0.001$ (derivative gain)
- Maximum $\alpha = 0.40$ (saturation limit)
- Integral windup protection: $|\int e(\tau) d\tau| \leq 0.20$

4.3.3 Steering Application

At each generation step, we:

1. Update chunk-level hidden state accumulation
2. When chunk is complete, classify redundancy level
3. Update PID controller based on redundancy error
4. Apply control vector with strength $\alpha(t)$ to current hidden states
5. Adjust sampling temperature based on steering strength

4.3.4 Adaptive Temperature Control

To complement activation steering, we dynamically adjust sampling temperature:

$$T(t) = T_{\text{base}} \times \left(1 - \frac{\alpha(t)}{\alpha_{\text{max}}}\right) + T_{\text{steer}} \times \frac{\alpha(t)}{\alpha_{\text{max}}} \quad (3)$$

Where $T_{\text{base}} = 0.60$ and $T_{\text{steer}} = 0.30$, making generation more focused when steering is active.

4.4 Generation Pipeline

The complete RASPID generation process:

1. **Initialization:** Set PID state variables to zero
2. **Free reasoning period:** Allow unrestricted generation for initial tokens
3. **Active steering period:** Apply PID-controlled steering for middle section
4. **Conclusion period:** Disable steering to allow natural completion
5. **Early stopping:** Terminate on answer patterns or excessive repetition

This three-phase approach ensures the model can begin reasoning naturally, receives guidance during the middle phase where redundancy typically occurs, and can conclude naturally.

5 Experimental Setup

5.1 Models and Datasets

We evaluate RASPID using:

- **Model:** DeepSeek-R1-Distill-Qwen-1.5B, a reasoning-capable open-source model
- **Training Data:** 1000 manually labeled reasoning chains from GSM8K problems
- **Evaluation Data:** 50 held-out GSM8K problems (problems 1000-1050)

5.2 Baselines

We compare against:

1. **Baseline:** Standard generation with temperature 0.6
2. **Prompt Engineering:** Direct instructions for brevity
3. **Static Steering:** Fixed activation steering without adaptive control

5.3 Evaluation Metrics

- **Accuracy:** Percentage of problems solved correctly
- **Token Efficiency:** Reduction in average tokens per problem
- **Answer Quality:** Manual assessment of reasoning quality

5.4 Implementation Details

- Chunk size: 24 tokens (best performing)
- Classifier validation accuracy: 72%
- Free reasoning period: 80 tokens
- Active steering window: 60 tokens
- Maximum generation length: 4096 tokens

6 Results

6.1 Main Results

Table ?? presents our main experimental results comparing RASPID against baselines on 50 GSM8K problems.

Method	Accuracy	Avg Tokens	Token Reduction
Baseline	58%	1744.4	—
Prompt Engineering	52%	1680.2	3.7%
Static Steering	54%	1456.8	16.5%
RASPID	64%	1182.8	32.2%

Table 1: Main experimental results on GSM8K problems.

RASPID achieves the best performance across all metrics, improving accuracy by 6 percentage points while reducing token usage by 32%.

6.2 Ablation Studies

6.2.1 PID Component Analysis

We analyze the contribution of each PID component:

Configuration	Accuracy	Token Reduction
P-only	60%	28.1%
PI	62%	30.4%
PD	61%	29.7%
PID (Full)	64%	32.2%

Table 2: Ablation study of PID components.

The full PID controller provides the best balance of accuracy and efficiency.

Classifier Accuracy	RASPID Accuracy	Token Reduction
60%	59%	25.1%
72%	64%	32.2%
85% (Oracle)	68%	38.5%

Table 3: Impact of classifier accuracy on RASPID performance.

6.2.2 Classifier Accuracy Impact

We evaluate how classifier performance affects overall results:

Better classifier accuracy consistently improves RASPID performance, suggesting potential for further improvements with enhanced classification.

6.3 Qualitative Analysis

Figure ?? shows a real-time trace of RASPID’s control behavior during generation. The system successfully identifies periods of high redundancy and applies appropriate steering while leaving useful reasoning unperturbed.

Example traces demonstrate RASPID’s ability to:

- Prevent excessive verification loops
- Reduce verbose explanations while preserving key steps
- Maintain mathematical accuracy despite aggressive pruning

6.4 Comparison with SEAL

While direct comparison is limited by different experimental setups, our results suggest RASPID provides complementary benefits to SEAL’s approach:

- **Adaptivity:** RASPID’s dynamic control vs. SEAL’s static coefficients
- **Granularity:** Real-time chunk-level feedback vs. predefined thought categories
- **Control Theory:** Principled PID design vs. heuristic steering

7 Analysis and Discussion

7.1 Why PID Control Works

The success of PID control in RASPID can be attributed to several factors:

1. **Proportional Response:** Immediate correction based on current redundancy level
2. **Integral Action:** Correction for persistent redundancy patterns
3. **Derivative Action:** Anticipation of redundancy trends
4. **Stability:** Established control theory ensures stable behavior

7.2 Real-Time Control Behavior

Analysis of RASPID’s real-time behavior reveals interesting patterns:

- **Early Phase:** Low steering as model establishes problem context
- **Middle Phase:** Increased steering during solution exploration
- **Late Phase:** Reduced steering as model converges to answer

This natural adaptation aligns with human problem-solving patterns.

7.3 Limitations

Several limitations of our current approach warrant discussion:

1. **Classifier Dependency:** Performance is bounded by classifier accuracy
2. **Domain Specificity:** Training on math problems may not generalize to other domains
3. **Hyperparameter Sensitivity:** PID gains require careful tuning
4. **Computational Overhead:** Real-time classification adds modest inference cost

7.4 Failure Cases

RASPID occasionally fails when:

- The classifier misidentifies useful reasoning as redundant
- Complex problems require more exploration than the controller permits
- PID parameters are poorly tuned for specific problem types

7.5 Broader Implications

Our work suggests several broader implications for LLM efficiency:

1. **Dynamic Control:** Adaptive intervention outperforms static approaches
2. **Real-Time Feedback:** Online redundancy detection enables fine-grained control
3. **Control Theory:** Classical control principles apply effectively to language generation
4. **Training-Free Methods:** Significant improvements possible without model retraining

8 Future Work

Several promising directions for future research emerge from this work:

8.1 Enhanced Classification

- **Multi-class Redundancy:** Distinguishing different types of redundant reasoning
- **Contextual Awareness:** Incorporating problem difficulty and domain context
- **Self-Supervised Learning:** Reducing dependence on manual labeling

8.2 Advanced Control Methods

- **Model Predictive Control:** Optimizing steering over future horizons
- **Adaptive Control:** Automatically tuning PID parameters during generation
- **Multi-Objective Control:** Balancing accuracy, efficiency, and other metrics

8.3 Broader Applications

- **Domain Transfer:** Extending to coding, scientific reasoning, and other domains
- **Model Scaling:** Evaluating effectiveness on larger reasoning models
- **Integration:** Combining with other efficiency techniques like speculative decoding

8.4 Theoretical Understanding

- **Controllability Analysis:** Formal characterization of reasoning steering capabilities
- **Stability Guarantees:** Theoretical bounds on PID controller behavior
- **Optimality:** Investigating optimal control formulations for reasoning efficiency

9 Conclusion

We have presented RASPID, a novel approach to improving the efficiency of reasoning-capable language models through dynamic activation steering with PID control. Our method addresses the critical problem of redundant token generation that plagues current reasoning models, achieving substantial token reductions while maintaining or improving accuracy.

The key innovations of RASPID include real-time redundancy detection through chunk-level classification, principled dynamic control using PID feedback, and a training-free deployment strategy that can be immediately applied to existing models. Our experimental results demonstrate the effectiveness of this approach, achieving 32% token reduction with 6% accuracy improvement on GSM8K mathematical reasoning tasks.

Beyond the immediate practical benefits, RASPID opens new research directions at the intersection of control theory and language model optimization. The success of classical control techniques in this domain suggests broader opportunities for applying established engineering principles to AI system optimization.

As reasoning-capable LLMs become increasingly important for complex problem-solving applications, methods like RASPID will be essential for making these powerful models practical and cost-effective for real-world deployment. Our work provides a foundation for future research in dynamic, adaptive control of language model behavior.

References

- Chen, R., Zhang, Z., Hong, J., Kundu, S., Wang, Z. SEAL: Steerable Reasoning Calibration of Large Language Models for Free. *Preprint*, 2025.
- Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Guo, D., Yang, D., Zhang, H., Song, J., Zhang, R., Xu, R., et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- Hendrycks, D., Burns, C., Kadavath, S., Arora, A., Basart, S., Tang, E., Song, D., Steinhardt, J. Measuring mathematical problem solving with the math dataset. *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2021.

- OpenAI. Learning to Reason with LLMs. 2024. URL <https://openai.com/index/learning-to-reason-with-llms>.
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q. V., Zhou, D., et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.
- Zou, A., Phan, L., Chen, S., Campbell, J., Guo, P., Ren, R., Pan, A., Yin, X., Mazeika, M., Dombrowski, A., et al. Representation engineering: A top-down approach to ai transparency. *arXiv preprint arXiv:2310.01405*, 2023.